

UNIVERSIDAD TECNOLÓGICA del Estado de Querétaro

División de Tecnologías de la Información y Comunicación

Práctica 1

OWASP API Security Top 10: Investigación y análisis de vulnerabilidades

Nombre del alumno: Saul Noe Perez Aguilera

Materia: Seguridad en el desarrollo de aplicaciones

20 de mayo de 2026

1. Tabla Comparativa: OWASP API Security Top 10 (2023)

A continuación se presenta la tabla comparativa enfocada específicamente en los riesgos de seguridad para APIs (OWASP API Security Top 10 2023).

Vulnerabilidad	Descripción Corta	Ejemplo de Ataque	Impacto Potencial	Método de Mitigación
API1:2023 Broken Object Level Authorization	Falta de validación de permisos al acceder a un objeto mediante su identificador (ID).	Un usuario cambia el parámetro <code>/api/users/12</code> por <code>/api/users/13</code> y accede a los datos de otra persona.	Exposición o manipulación no autorizada de datos de otros usuarios (robo de información).	Validar siempre en el backend que el usuario autenticado tiene permisos sobre el ID solicitado.
API2:2023 Broken Authentication	Mecanismos de autenticación implementados incorrectamente que permiten comprometer tokens o identidades.	Un atacante roba un token JWT mal firmado o explota una API de recuperación de contraseña débil.	Toma de control de cuentas de usuarios, permitiendo suplantación de identidad temporal o permanente.	Usar estándares robustos (OAuth2, OIDC), exigir MFA y gestionar correctamente el ciclo de vida de tokens.
API3:2023 Broken Object Property Level Auth	Exposición excesiva de datos o asignación masiva por falta de validación de permisos a nivel de propiedad.	Enviar un JSON actualizando el perfil incluyendo <code>is_admin": true</code> y que la API lo guarde sin validar.	Escalación de privilegios, manipulación de estados internos y fuga de información sensible.	Validar explícitamente los campos que el usuario puede leer o modificar; evitar la asignación automática (binding).
API4:2023 Unrestricted Resource Consumption	Consumo de recursos de la API (CPU, memoria, SMS de terceros) sin límites establecidos.	Un atacante realiza miles de peticiones por segundo al endpoint de "Enviar SMS de validación".	Denegación de Servicio (DoS) y aumento masivo de costos operativos de facturación en la nube.	Implementar <i>Rate Limiting</i> (limitación de tasa), cuotas por usuario y alertas de facturación.
API5:2023 Broken Function Level Authorization	Falta de separación clara entre funciones regulares y administrativas, permitiendo acceso indebido.	Un usuario con rol básico envía un método DELETE a <code>/api/v1/admin/users/5</code> y el sistema lo procesa.	Acceso a funciones críticas, modificación o borrado de recursos a nivel administrador.	Implementar control de acceso basado en roles (RBAC) estricto y denegar funciones por defecto.
API6:2023 Unrestricted Access to Sensitive Flows	Exposición de flujos de negocio (compras, comentarios) sin protección contra el uso automatizado.	Un script automatizado (bot) compra en segundos todo el inventario de boletos disponibles en la API.	Daño directo al negocio, fraude o acaparamiento de recursos legítimos (scalping).	Identificar flujos sensibles y usar mecanismos anti-bots (captchas, análisis de comportamiento).

Vulnerabilidad	Descripción Corta	Ejemplo de Ataque	Impacto Potencial	Método de Mitigación
API7:2023 Server Side Request Forgery (SSRF)	La API obtiene un recurso remoto a partir de una URI dada por el usuario sin validarla.	Un atacante envía un webhook con la URL <code>http://169.254.169.254</code> para extraer credenciales en la nube.	Acceso a la red interna de la empresa, derivación de <code>ip:254</code> y exposición de metadatos sensibles.	Validar URIs con listas blancas (allow-lists), deshabilitar redirecciones y aislar la red de la API.
API8:2023 Security Misconfiguration	Configuraciones de seguridad deficientes, mensajes de error detallados o configuraciones por defecto.	La API responde con un "stack trace" completo (rutas de carpetas y versiones de software) ante un error 500.	Exposición de información valiosa para preparar ataques más sofisticados contra la infraestructura.	Establecer procesos de <i>hardening</i> , auditar configuraciones CORS y enviar errores genéricos al cliente.
API9:2023 Improper Inventory Management	Falta de inventario y documentación de las APIs, dejando endpoints de prueba o versiones antiguas expuestas.	Un atacante descubre y explota <code>/api/v1/users</code> (vulnerable) porque la empresa solo actualizó <code>/api/v2/users</code> .	Explotación de vulnerabilidades ya corregidas en versiones nuevas y acceso a datos mediante "shadow APIs".	Mantener documentación viva (ej. Swagger/OpenAPI) y desmantelar (retirar) las versiones antiguas de la API.
API10:2023 Unsafe Consumption of APIs	Confianza excesiva en los datos recibidos desde APIs de terceros, adoptando estándares de seguridad débiles.	Un atacante compromete una API de clima de terceros, inyectando SQL que luego es ejecutado por tu propia API.	Compromiso total de la aplicación a través de un vector de ataque indirecto (cadena de suministro de datos).	Tratar los datos de APIs de terceros como "no confiables", validándolos y sanitizándolos igual que al usuario.

2. Casos Reales de Vulnerabilidades en APIs

A continuación se documentan incidentes reales correspondientes a las vulnerabilidades del OWASP API Security Top 10 (2023).

API1:2023 - Broken Object Level Authorization (BOLA)

- **Empresa afectada:** Peloton
- **Año:** 2021
- **Cómo fue explotada:** Investigadores de seguridad descubrieron que la API de la aplicación permitía realizar consultas sin autenticación. Modificando los ID de usuario en las peticiones HTTP, cualquier persona podía acceder a los datos privados del perfil de otros clientes (edad, peso, ciudad, historial de ejercicios), incluso si sus cuentas estaban configuradas como "privadas".
- **Empresa afectada:** Verizon
- **Año:** 2022
- **Cómo fue explotada:** Un analista de ciberseguridad demostró que la API de la plataforma cliente no validaba la propiedad del objeto. Al interceptar el tráfico web e inyectar el número de contrato de otra persona en el parámetro de la petición, el sistema devolvía los datos completos del otro usuario.

API2:2023 - Broken Authentication

- **Empresa afectada:** Optus (Operadora de telecomunicaciones)
- **Año:** 2022
- **Cómo fue explotada:** Uno de los mayores ciberataques en Australia ocurrió cuando dejaron expuesto un endpoint de API en internet que no requería ningún tipo de autenticación (ni token, ni contraseña). Los atacantes simplemente iteraron sobre la API para extraer los datos personales de casi 10 millones de clientes.
- **Empresa afectada:** T-Mobile
- **Año:** 2023
- **Cómo fue explotada:** Una falla crítica en los controles de autenticación de una de sus APIs permitió a actores maliciosos extraer la información de 37 millones de cuentas durante meses. La API no manejó correctamente el ciclo de vida y alcance de los tokens, permitiendo accesos continuos y no autorizados.

API3:2023 - Broken Object Property Level Authorization (BOPLA)

- **Empresa afectada:** Twitter (ahora X)
- **Año:** 2022
- **Cómo fue explotada:** Una API utilizada para el inicio de sesión y recuperación de cuentas sufría de "Exposición excesiva de datos". Al consultarla con un número de teléfono o ID, la API devolvía en el JSON de respuesta toda la estructura interna de la cuenta, filtrando millones de correos y números de teléfono asociados, a pesar de que la interfaz gráfica no mostraba esa información.

- **Empresa afectada:** Booking.com
- **Año:** 2023
- **Cómo fue explotada:** Investigadores encontraron fallas de asignación masiva y validación de propiedades. Al modificar campos de estado en el payload JSON durante el flujo de creación de cuenta y configuración, lograron manipular los estados de la base de datos para comprometer cuentas y elevar privilegios.

API4:2023 - Unrestricted Resource Consumption

- **Empresa afectada:** Ticketmaster
- **Año:** 2022
- **Cómo fue explotada:** Durante la venta de boletos para el tour de Taylor Swift, una legión masiva de bots automatizados atacó la API de pago sin restricciones de límite de tasa (*rate limiting*). La avalancha de peticiones provocó una denegación de servicio (DoS) a nivel de aplicación, tumbando el sistema e impidiendo la compra a usuarios legítimos.
- **Empresa afectada:** GitHub
- **Año:** 2021
- **Cómo fue explotada:** Atacantes se aprovecharon de que las APIs de *GitHub Actions* no limitaban correctamente los recursos computacionales por usuario. Crearon scripts que generaban pull requests masivos utilizando la API para ejecutar mineros de criptomonedas en los servidores de GitHub sin costo.

API5:2023 - Broken Function Level Authorization (BFLA)

- **Empresa afectada:** Múltiples fabricantes de automóviles (Nissan, BMW, Honda)
- **Año:** 2022
- **Cómo fue explotada:** Se encontraron endpoints de administración en los sistemas telemáticos de los vehículos. Las APIs no validaban el rol del usuario que emitía la orden, permitiendo a cuentas estándar invocar funciones administrativas como desbloquear puertas, encender el motor y transferir la propiedad del vehículo.
- **Empresa afectada:** Shopify
- **Año:** 2020
- **Cómo fue explotada:** Un cazarrecompensas descubrió que podía asignar permisos de administrador a un usuario común, forzando peticiones POST hacia funciones internas del sistema (ej. `api/admin/users`) que no validaban adecuadamente el origen jerárquico del token de sesión.

API6:2023 - Unrestricted Access to Sensitive Business Flows

- **Empresa afectada:** Tiendas minoristas de tecnología (Walmart, BestBuy)
- **Año:** 2020-2021
- **Cómo fue explotada:** Redes de revendedores (*scalpers*) identificaron el flujo de negocio del carrito de compras para las consolas PS5. Mediante secuencias de comandos, interactuaban con la API de inventario para automatizar el ciclo de reserva y pago en milisegundos, saltándose por completo la cola de clientes regulares.

- **Empresa afectada:** Parler (Red social)
- **Año:** 2021
- **Cómo fue explotada:** La plataforma carecía de mitigaciones contra la automatización y la enumeración de su API. Previo a su cierre de servidores, activistas utilizaron scripts simples que automatizaron la descarga (*scraping*) de terabytes de publicaciones, videos con metadatos y ubicaciones GPS, dañando la privacidad del negocio.

API7:2023 - Server Side Request Forgery (SSRF)

- **Empresa afectada:** Capital One
- **Año:** 2019
- **Cómo fue explotada:** Un atacante manipuló una API vulnerable al SSRF enviándole como parámetro la dirección interna de los servicios de metadatos de AWS (<http://169.254.169.254>). El servidor procesó la solicitud asumiendo que era de confianza, y le devolvió credenciales maestras de la nube, comprometiendo 106 millones de registros bancarios.
- **Empresa afectada:** Microsoft Exchange
- **Año:** 2021
- **Cómo fue explotada:** Una vulnerabilidad SSRF en la API de Exchange permitió a atacantes no autenticados enviar solicitudes HTTP fabricadas, engañando al servidor para que procesara y reenviara conexiones maliciosas hacia la red interna como si el propio servidor las hubiera iniciado.

API8:2023 - Security Misconfiguration

- **Empresa afectada:** Twitch
- **Año:** 2021
- **Cómo fue explotada:** Una configuración defectuosa en los repositorios de la nube vinculados a sus APIs derivó en la exposición pública del código fuente completo de la plataforma, así como en la fuga masiva de los reportes de ingresos y comisiones pagadas a los streamers.
- **Empresa afectada:** Miles de aplicaciones móviles (Uso de Firebase)
- **Año:** 2020
- **Cómo fue explotada:** Desarrolladores configuraron erróneamente las reglas de seguridad de sus APIs de base de datos en Firebase, dejándolas abiertas al público (`read: true`, `write: true`). Esto permitió extraer información personal y médica de miles de apps sin el uso de exploits complejos.

API9:2023 - Improper Inventory Management

- **Empresa afectada:** JustDial
- **Año:** 2019
- **Cómo fue explotada:** La empresa lanzó versiones seguras de su API, pero olvidó dismantelar una versión antigua no documentada ("Shadow API"). Los atacantes la descubrieron y, como aún tenía acceso a la base de datos productiva, lograron exponer 100 millones de perfiles de clientes.

- **Empresa afectada:** Experian (Buró de crédito)
- **Año:** 2021
- **Cómo fue explotada:** Un endpoint de API que fue creado para un socio comercial específico fue olvidado en producción sin controles de seguridad. Con tan solo enviar nombres y domicilios, la API permitía consultar el historial crediticio de casi cualquier ciudadano.

API10:2023 - Unsafe Consumption of APIs

- **Empresa afectada:** MOVEit (Software de transferencia de archivos)
- **Año:** 2023
- **Cómo fue explotada:** Las compañías que consumían la API de MOVEit fueron vulneradas debido a que el software confiaba ciegamente en parámetros provenientes de flujos externos. La falta de sanitización de las entradas provocó una inyección SQL que desencadenó el robo masivo de información.
- **Empresa afectada:** Ecosistema global Java (Log4Shell)
- **Año:** 2021
- **Cómo fue explotada:** Las APIs consumían encabezados web enviados por terceros sin validarlos y los pasaban a la biblioteca de registro Log4j. Esta confianza excesiva hacia el input recibido del exterior resultó en el compromiso remoto de infraestructuras críticas en todo el mundo.

3. Ejemplos de Código: Vulnerable vs. Seguro

A continuación se presentan tres escenarios implementados en JavaScript (Node.js / Express) que exponen la fragilidad del backend cuando se omiten validaciones estructurales de control de acceso y sanitización de datos.

1. API1:2023 - Broken Object Level Authorization (BOLA)

Esta falla se origina cuando el endpoint procesa un identificador de objeto confiando ciegamente en el cliente sin comprobar la jerarquía del token autenticado sobre dicho objeto.

```
1 // El endpoint obtiene el perfil usando el ID de la URL, pero no valida si
  // el usuario logueado es el dueño del recurso
2 app.get('/api/v1/profile/:id', verifyToken, async (req, res) => {
3   const userProfile = await User.findById(req.params.id);
4   res.json(userProfile);
5 });
```

Listing 1: Código Vulnerable: Acceso directo al recurso por ID sin verificar pertenencia.

```
1 app.get('/api/v1/profile/:id', verifyToken, async (req, res) => {
2   // Validacion: Se compara el ID del token verificado (req.user.id) con
  // el parametro de la ruta
3   if (req.user.id !== req.params.id) {
4     return res.status(403).json({ error: "Acceso denegado: No posees
      permisos sobre este perfil." });
5   }
6   const userProfile = await User.findById(req.params.id);
7   res.json(userProfile);
8 });
```

Listing 2: Código Seguro: Validación estricta cruzada del contexto de sesión.

2. API3:2023 - Broken Object Property Level Authorization

Ocurre cuando la API procesa payloads JSON complejos directamente en sentencias de actualización de base de datos sin aplicar un filtrado explícito de parámetros permitidos (*Mass Assignment*).

```
1 // Guarda directamente todo el cuerpo de la peticion sin discriminar
  // propiedades internas
2 // Un atacante podria inyectar {"is_admin": true} y elevar privilegios
3 app.put('/api/v1/users/:id', verifyToken, async (req, res) => {
4   const updatedUser = await User.findByIdAndUpdate(req.params.id, req.body
5     , { new: true });
6   res.json(updatedUser);
7 });
```

Listing 3: Código Vulnerable: Enlace directo de payloads modificables.

```
1 app.put('/api/v1/users/:id', verifyToken, async (req, res) => {
2   // Uso de una lista blanca: Se extraen explícitamente solo las
  // propiedades seguras
3   const { name, email, age } = req.body;
4
5   // Cualquier propiedad no declarada arriba (ej. roles) es ignorada por
  // el motor ORM
6   const updatedUser = await User.findByIdAndUpdate(
7     req.params.id,
```



```

8      { name, email, age },
9      { new: true }
10   );
11   res.json(updatedUser);
12 });

```

Listing 4: Código Seguro: Implementación de listas blancas en desestructuración.

3. API10:2023 - Unsafe Consumption of APIs

Esta vulnerabilidad expone la tendencia a relajar los estándares de sanitización de entradas cuando provienen de integraciones y microservicios externos considerados “de confianza”.

```

1 app.post('/api/v1/sync-weather', async (req, res) => {
2   const response = await fetch('https://api.clima-tercero.com/data');
3   const weatherData = await response.json();
4   const cityName = weatherData.city; // Entrada de datos externa no
      sanitizada
5
6   // Inyeccion SQL: Concatenacion cruda sobre la cadena de comandos
7   await db.query('INSERT INTO weather_logs (city) VALUES ('${cityName}')');
8   res.send("Datos sincronizados");
9 });

```

Listing 5: Código Vulnerable: Concatenación directa de entradas de terceros.

```

1 app.post('/api/v1/sync-weather', async (req, res) => {
2   const response = await fetch('https://api-clima-tercero.com/data');
3   const weatherData = await response.json();
4   const cityName = weatherData.city;
5
6   // Mitigacion: Uso mandatorio de consultas preparadas, neutralizando
      cualquier exploit
7   await db.query('INSERT INTO weather_logs (city) VALUES (?)', [cityName]);
8   res.send("Datos sincronizados");
9 });

```

Listing 6: Código Seguro: Aislamiento mediante consultas parametrizadas.

4. Ranking Personal de Vulnerabilidades Críticas

A continuación, presento mi ranking personal de las 5 vulnerabilidades más críticas del OWASP API Security Top 10, basándome en mi criterio sobre la facilidad de explotación y el impacto en arquitecturas modernas de desarrollo:

1. **API1:2023 Broken Object Level Authorization (BOLA)**

La considero la más crítica porque la manipulación de identificadores (IDs) en las rutas web es el vector de ataque más accesible e intuitivo. Como desarrollador, entiendo que es sumamente fácil cometer el error de no validar cruzadamente si el usuario en sesión es el propietario real del recurso en el backend.

2. **API2:2023 Broken Authentication**

Sin una autenticación sólida, el resto de las defensas del sistema se vuelven inútiles. Tomando en cuenta la importancia de la criptografía, un manejo deficiente de claves o el uso de tokens JWT mal firmados permiten a los atacantes comprometer la infraestructura desde la puerta de entrada principal.

3. **API10:2023 Unsafe Consumption of APIs**

En el desarrollo full-stack actual (usando entornos como Node.js o React), los proyectos dependen masivamente de microservicios e integraciones de terceros. Confiar ciegamente en estas respuestas externas sin desinfectarlas introduce riesgos de inyección y fallos severos en la cadena de suministro.

4. **API3:2023 Broken Object Property Level Authorization**

Es una práctica muy común en frameworks modernos mapear directamente los objetos JSON recibidos hacia los modelos de la base de datos para ahorrar tiempo. Si no establecemos listas blancas estrictas, basta con que un atacante inyecte una propiedad extra para escalar sus privilegios.

5. **API7:2023 Server Side Request Forgery (SSRF)**

La ubico en este top porque la mayoría de las aplicaciones hoy en día se despliegan en la nube. Un ataque SSRF es devastador en estos entornos, ya que permite saltarse los firewalls perimetrales para interactuar con servicios internos y extraer metadatos altamente confidenciales.

5. Conclusión

La seguridad en el desarrollo de software ha dejado de ser una responsabilidad exclusiva de los equipos de operaciones o de los administradores de sistemas para convertirse en un pilar fundamental dentro de las competencias de todo desarrollador de software. La asimilación de estándares como el OWASP API Security Top 10 representa un cambio de paradigma crucial en la forma en que concebimos, estructuramos y escribimos código en el día a día. Al comprender la naturaleza profunda de riesgos críticos como la autorización rota a nivel de objeto o el consumo desmedido de recursos, dejamos de diseñar aplicaciones basándonos puramente en la funcionalidad para empezar a construir arquitecturas resilientes capaces de soportar intentos reales de explotación maliciosa en entornos productivos.

En mi práctica profesional como ingeniero de software, planeo aplicar este conocimiento de manera transversal a lo largo de todo el ciclo de vida del desarrollo. Esto significa adoptar una postura proactiva y defensiva desde las etapas de diseño y modelado de amenazas, en lugar de intentar parchar vulnerabilidades de forma reactiva cuando el sistema ya se encuentra desplegado en producción. En el desarrollo full-stack, implementaré de forma mandatoria la sanitización rigurosa y la parametrización de consultas para evitar inyecciones, la desestructuración de payloads para anular la asignación masiva de propiedades no deseadas, y la validación cruzada y estricta en el servidor de los tokens de sesión frente a cada identificador de recurso solicitado. Asimismo, integraré herramientas de escaneo automatizado en los pipelines de integración continua para monitorear dependencias vulnerables y asegurar configuraciones óptimas en la nube. En conclusión, dominar el OWASP Top 10 no solo mitiga riesgos financieros y legales devastadores para las organizaciones, sino que eleva la calidad integral del producto de software, protegiendo el activo más valioso en la actualidad: la integridad y privacidad de la información de los usuarios.

Fuentes de información:

- Fundación OWASP. *OWASP API Security Top 10 - 2023*. Disponible en: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- Krebs on Security, The Guardian, Salt Security Labs y cvedetails.com (Reportes de vulnerabilidades públicas y brechas de datos).